

CS-UY 2214 — Homework 5

Adapted from Jeff Epstein, Modified by Ratan Dey

Introduction

Unless otherwise specified, put your answers in a plain text file named **hw5.txt**. Number each answer. Submit your work on Gradescope.

You may consult the E20 manual, which is available on Brightspace.

Problems

1. Write an E20 assembly language program that will store the value 1099 at memory cell 456, then halt. Your solution should consist of no more than seven instructions. Optional: you may use your E20 assembler `conecpt` (translating E20 assembly instructions to E20 Machine Codes manually following E20 manual) to make sure that your program is correct and can be assembled into valid machine code.
2. Write an E20 assembly language program that will read the values of memory cells 20 and 30, and store their sum into memory cell 40. Don't forget to include a `halt` at the end.

Give your answer only in E20 assembly language (using correct E20 syntax).

3. Write an E20 assembly language program that will add 1 to each of the memory cells between 10 and 20, inclusive. Use a loop. Don't forget to include a `halt` at the end. When using jump instructions, specify the destination as a label, not as a numeric address.

Translate your E20 assembly language program into the equivalent machine language representation. Give each instruction as a 16-bit binary number. Write each instruction on a separate line. Write all 16 digits of each number.

Give your answer both in E20 assembly language (using correct E20 syntax) and E20 machine language (as a sequence of 16-bit binary numbers).

4. You are given a partial E20 assembly language program containing only the following:

```
beginning:
    .fill 3
    .fill 1
    .fill 0
    .fill 9
    .fill 5
    .fill 7
    .fill 1
    .fill 4
end:
```

That is, your program's data consists of an array of numbers between addresses **beginning** (inclusive) and **end** (exclusive).

Complete the program so that it reverses the contents of the array in place. In other words, when the program ends, the value at memory address **beginning** will be 4; the value at memory address **beginning+1** will be 1; the value at **beginning+2** will be 7; the value at memory address **end-1** will be 3; etc.

You must implement your program using a loop. Your program should work with an array of any length. You may not assume that the length or content of the array is the same as given above. The only thing you know about the array is that it is bound by labels **beginning** and **end**.

When using jump instructions, specify the destination as a label, not as a numeric address.

Hint: use paired **lw** and **sw** to copy a memory value from one address to another.